Stable matching

Pierre Pinson

Imperial College London, Dyson School of Design Engineering
mail: p.pinson@imperial.ac.uk - **webpage:** www.pierrepinson.com

Game Theory and Mechanism Design - Lecture 14

Learning objectives

At the end of this lecture (and related tutorial(s)), a student should be able to:

- Understand why matching is often referred to as **mechanism design without money**
- Describe **examples** of matchings problems
- Discuss what makes **"good" matching**
- Apply relevant **matching algorithms**

Outline

1. Matching problems
2. Stability
3. Algorithms and desirable properties

1. **Matching problems**

IT/Comms resources

- There a set of satellites (with different access costs, performance, etc.) that can be used for IT/Comms purposes
- Satellites (or group of) are operated independently of each other
- Ground-based systems are competing to have access to such satellites
- They are also operated on an independent and competitive basis

How to best connect ground-based systems with satellites?

[source: livescience.com/internet]

The original mating problem

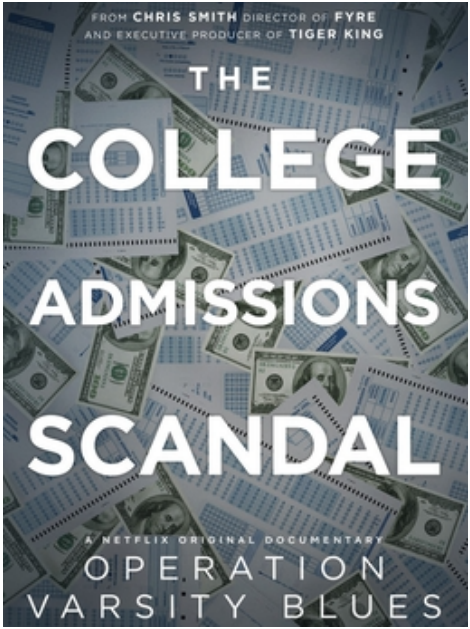
[source: filmvestigate.blog]

This is one of the most common application considered to describe matching problems...

- Consider 2 groups of n individuals who may be interested in dating someone from the other group.
- They are all selfish and rational
- They all express preferences about who they would prefer to date

How to best match individuals from these 2 groups?

The twist: College admissions



[source: en.wikipedia.org]

This is also one of the most common application considered to describe matching problems...

- A large number of prospective students (could be even worldwide) express preferences to join certain colleges (/universities)
- They are all selfish and rational (both prospective students and colleges)
- Colleges also rank prospective students through their admission process

How to best match students with colleges?

An obvious alternative version of this problem is the MEng final project matching problem...

The general case

It is about finding the best way to connect elements of 2 groups.

Definition (matching problem)

Given

- 2 sets $\mathcal{O} = \{o_1, o_2, \ldots, o_m\}$ and $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$ of cardinality m and n, respectively,
- preferences expressed by each element of $\mathcal{O}$ regarding elements of $\mathcal{D}$,
- preferences expressed by each element of $\mathcal{D}$ regarding elements of $\mathcal{O}$,

The *matching problem* consists in finding, among all potential matching alternatives, the one that makes most sense for all elements involved.

The general case

It is about finding the best way to connect elements of 2 groups.

Definition (matching problem)

Given

- 2 sets $\mathcal{O} = \{o_1, o_2, \ldots, o_m\}$ and $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$ of cardinality m and n, respectively,
- preferences expressed by each element of $\mathcal{O}$ regarding elements of $\mathcal{D}$,
- preferences expressed by each element of $\mathcal{D}$ regarding elements of $\mathcal{O}$,

The *matching problem* consists in finding, among all potential matching alternatives, the one that makes most sense for all elements involved.

To simplify things, we focus on a (slightly) simplified version.

Definition (marriage problem)

A *marriage problem* is a matching problem for which the two sets $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$ and $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$ have the same cardinality n.

There are $n! = n \times (n-1) \times \ldots \times 1$ potential combinations... (exponential growth with dimensionality)

A working example

Let us consider 2 groups of 4 people (say, 4 students and 4 teachers to be matched for MEng final projects), i.e.

- $\mathcal{O} = \{o_1, o_2, o_3, o_4\}$ (students)
- $\mathcal{D} = \{d_1, d_2, d_3, d_4\}$ (teachers)

Their preferences are defined as:

Student side:

	1	2	3	4
o_1	d_4	d_3	d_2	d_1
o_2	d_1	d_4	d_3	d_2
o_3	d_1	d_2	d_3	d_4
o_4	d_4	d_1	d_2	d_3

Teacher side:

	1	2	3	4
d_1	o_1	o_4	o_3	o_2
d_2	o_3	o_4	o_2	o_1
d_3	o_3	o_2	o_1	o_4
d_4	o_3	o_4	o_2	o_1

For instance,

- for student o_2: $d_1 \succ d_4 \succ d_3 \succ d_2$
- for teacher d_1: $o_1 \succ o_4 \succ o_3 \succ o_2$

Perfect matching (or allocation)

Solving the matching problem yields a matching (or allocation)

Definition (perfect matching (or allocation))

Given a marriage problem for 2 sets $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$ and $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$, and related expressed preferences, a *perfect matching* (or *allocation*) M consists in a proposal (and exclusive) association of each and every element of $\mathcal{O}$ with each and every element of $\mathcal{D}$. If writing ϕ the matching function, we have

- $M = \{(o_1, \phi(o_1)), \ldots, (o_n, \phi(o_n))\}$, or
- $M = \{(\phi^{-1}(d_1), d_1), \ldots, (\phi^{-1}(d_n), d_n)\}$

Perfect matching (or allocation)

Solving the matching problem yields a matching (or allocation)

Definition (perfect matching (or allocation))

Given a marriage problem for 2 sets $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$ and $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$, and related expressed preferences, a *perfect matching* (or *allocation*) M consists in a proposal (and exclusive) association of each and every element of $\mathcal{O}$ with each and every element of $\mathcal{D}$. If writing ϕ the matching function, we have

- $M = \{(o_1, \phi(o_1)), \ldots, (o_n, \phi(o_n))\}$, or
- $M = \{(\phi^{-1}(d_1), d_1), \ldots, (\phi^{-1}(d_n), d_n)\}$

For our working example...

Student side:

	1	2	3	4
o_1	d_4	d_3	d_2	d_1
o_2	d_1	d_4	d_3	d_2
o_3	d_1	d_2	d_3	d_4
o_4	d_4	d_1	d_2	d_3

Teacher side:

	1	2	3	4
d_1	o_1	o_4	o_3	o_2
d_2	o_3	o_4	o_2	o_1
d_3	o_3	o_2	o_1	o_4
d_4	o_3	o_4	o_2	o_1

An example allocation (perfect matching) is $M = \{(o_1, d_1), (o_2, d_2), (o_3, d_4), (o_4, d_4)\}$

2. **Stability**

Unstable pair

We both agree we could have been matched better...

Definition (unstable pair)

Consider an allocation M (and related matching function ϕ), an *unstable pair* (o_i, d_j) is a pair of elements from $\mathcal{O}$ and $\mathcal{D}$ such that

- for o_i, $d_j \succ \phi(o_i)$
- for d_j, $o_i \succ \phi^{-1}(d_j)$

In other words, they both agree they would prefer to be matched together.

Unstable pair

We both agree we could have been matched better...

Definition (unstable pair)

Consider an allocation M (and related matching function ϕ), an *unstable pair* (o_i, d_j) is a pair of elements from $\mathcal{O}$ and $\mathcal{D}$ such that

- for o_i, $d_j \succ \phi(o_i)$
- for d_j, $o_i \succ \phi^{-1}(d_j)$

In other words, they both agree they would prefer to be matched together.

For our working example...

Student side:

	1	2	3	4
o_1	d_4	d_3	d_2	d_1
o_2	d_1	d_4	d_3	d_2
o_3	d_1	d_2	d_3	d_4
o_4	d_4	d_1	d_2	d_3

Teacher side:

	1	2	3	4
d_1	o_1	o_4	o_3	o_2
d_2	o_3	o_4	o_2	o_1
d_3	o_3	o_2	o_1	o_4
d_4	o_3	o_4	o_2	o_1

(o_3, d_2) is an unstable pair since

- for o_3, $d_2 \succ \phi(o_1)$ (where $\phi(o_3) = d_3$)
- for d_2, $o_3 \succ \phi^{-1}(d_2)$ (where $\phi^{-1}(d_2) = d_2$)

Stability

No pair of elements should feel they could have jointly been better off

Definition (stable matching)

Given a marriage problem for 2 sets $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$ and $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$, and related expressed preferences, a *stable matching* outcome is an allocation for which no unstable pair exists.

Stability

No pair of elements should feel they could have jointly been better off

Definition (stable matching)

Given a marriage problem for 2 sets $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$ and $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$, and related expressed preferences, a *stable matching* outcome is an allocation for which no unstable pair exists.

For our working example...

Student side:

	1	2	3	4
o_1	d_4	d_3	d_2	d_1
o_2	d_1	d_4	d_3	d_2
o_3	d_1	d_2	d_3	d_4
o_4	d_4	d_1	d_2	d_3

Teacher side:

	1	2	3	4
d_1	o_1	o_4	o_3	o_2
d_2	o_3	o_4	o_2	o_1
d_3	o_3	o_2	o_1	o_4
d_4	o_3	o_4	o_2	o_1

$M = \{(o_1, d_2), (o_2, d_3), (o_3, d_1), (o_4, d_4)\}$ is a stable matching!
(as I cannot find any unstable pair...)

Existence of a stable allocation

Who said it was always possible to find a stable allocation?

Proposition (existence of stable allocation)

Given a marriage problem for 2 sets $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$ and $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$, and related expressed preferences, there is no guarantee that a stable allocation exists.

Existence of a stable allocation

Who said it was always possible to find a stable allocation?

Proposition (existence of stable allocation)

Given a marriage problem for 2 sets $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$ and $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$, and related expressed preferences, there is no guarantee that a stable allocation exists.

A room-mate matching problem: each player ranks others

	1	2	3
A	B	C	D
B	C	A	D
C	A	B	D
D	A	B	C

Good luck... because not stable allocation exists here! See that

- $M = \{(A, B), (C, D)\}$ - then (B, C) is unstable
- $M = \{(A, D), (B, C)\}$ - then (A, C) is unstable
- etc.

Multiple stable allocations

There can be several stable allocations

Proposition (multiple stable allocations)

Given a marriage problem for 2 sets $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$ and $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$, and related expressed preferences, if a stable allocation exists, there may be more than one.

Multiple stable allocations

There can be several stable allocations

Proposition (multiple stable allocations)

Given a marriage problem for 2 sets $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$ and $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$, and related expressed preferences, if a stable allocation exists, there may be more than one.

A resource allocation problem, e.g., for satellite communication allocation:

Satellite side:

	1	2	3
o_1	d_1	d_2	d_3
o_2	d_2	d_1	d_3
o_3	d_1	d_2	d_3

Ground-network side:

	1	2	3
d_1	o_2	o_1	o_3
d_2	o_1	o_2	o_3
d_3	o_1	o_2	o_3

There are 2 stable allocations there:

- $M = \{(o_1, d_1), (o_2, d_2), (o_3, d_3)\}$ (in light grey)
- $M = \{(o_1, d_2), (o_2, d_1), (o_3, d_3)\}$ (in darker grey)

3. **Algorithms and desirable properties**

The Gale-Shapley algorithm

There may be other approaches, but this is the most popular algorithm

Definition (Gale-Shapley algorithm)

Given a marriage problem for 2 sets $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$ and $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$, and related expressed preferences, the Gale-Shapley algorithm proceeds as following:

1. each element of $\mathcal{O}$ proposes to their favourite element of $\mathcal{D}$
2. for each element of $\mathcal{D}$ receiving at least one proposal from $\mathcal{O}$, they pick their favourite one and place them in a queue, in case a preferred element of $\mathcal{O}$ comes at a later stage
3. the elements of $\mathcal{O}$ that got rejected proposes to their next favourite element of $\mathcal{D}$
4. for each element of $\mathcal{D}$ receiving at least one proposal from $\mathcal{O}$ (including the one previously accepted), they pick their favourite one and place them in a queue, in case a preferred element of $\mathcal{O}$ comes at a later stage
5. Go back to 3, until all elements of $\mathcal{D}$ have been proposed to...

The Gale-Shapley algorithm

There may be other approaches, but this is the most popular algorithm

Definition (Gale-Shapley algorithm)

Given a marriage problem for 2 sets $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$ and $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$, and related expressed preferences, the Gale-Shapley algorithm proceeds as following:

1. each element of $\mathcal{O}$ proposes to their favourite element of $\mathcal{D}$
2. for each element of $\mathcal{D}$ receiving at least one proposal from $\mathcal{O}$, they pick their favourite one and place them in a queue, in case a preferred element of $\mathcal{O}$ comes at a later stage
3. the elements of $\mathcal{O}$ that got rejected proposes to their next favourite element of $\mathcal{D}$
4. for each element of $\mathcal{D}$ receiving at least one proposal from $\mathcal{O}$ (including the one previously accepted), they pick their favourite one and place them in a queue, in case a preferred element of $\mathcal{O}$ comes at a later stage
5. Go back to 3, until all elements of $\mathcal{D}$ have been proposed to...

Is there anything that looks suspicious to you?

The Gale-Shapley algorithm

There may be other approaches, but this is the most popular algorithm

Definition (Gale-Shapley algorithm)

Given a marriage problem for 2 sets $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$ and $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$, and related expressed preferences, the Gale-Shapley algorithm proceeds as following:

1. each element of $\mathcal{O}$ proposes to their favourite element of $\mathcal{D}$
2. for each element of $\mathcal{D}$ receiving at least one proposal from $\mathcal{O}$, they pick their favourite one and place them in a queue, in case a preferred element of $\mathcal{O}$ comes at a later stage
3. the elements of $\mathcal{O}$ that got rejected proposes to their next favourite element of $\mathcal{D}$
4. for each element of $\mathcal{D}$ receiving at least one proposal from $\mathcal{O}$ (including the one previously accepted), they pick their favourite one and place them in a queue, in case a preferred element of $\mathcal{O}$ comes at a later stage
5. Go back to 3, until all elements of $\mathcal{D}$ have been proposed to...

Is there anything that looks suspicious to you?

This is the $\mathcal{O}$-version of the algorithm...
We can also define a $\mathcal{D}$-version of that same algorithm (with elements of $\mathcal{D}$ proposing in the first place)

Then... Let's do it!

$\mathcal{O}$-version

Half of you will apply this version of the algorithm and tell us what the resulting allocation is...

$\mathcal{D}$-version

The other half will apply this other version of the algorithm and tell us what the resulting allocation is...

Student side:

	1	2	3	4
o_1	d_4	d_3	d_2	d_1
o_2	d_1	d_4	d_3	d_2
o_3	d_1	d_2	d_3	d_4
o_4	d_4	d_1	d_2	d_3

Teacher side:

	1	2	3	4
d_1	o_1	o_4	o_3	o_2
d_2	o_3	o_4	o_2	o_1
d_3	o_3	o_2	o_1	o_4
d_4	o_3	o_4	o_2	o_1

Resulting allocations (any obvious observation?)

$\mathcal{O}$-version

Student side:

	1	2	3	4
o_1	d_4	d_3	d_2	d_1
o_2	d_1	d_4	d_3	d_2
o_3	d_1	d_2	d_3	d_4
o_4	d_4	d_1	d_2	d_3

Teacher side:

	1	2	3	4
d_1	o_1	o_4	o_3	o_2
d_2	o_3	o_4	o_2	o_1
d_3	o_3	o_2	o_1	o_4
d_4	o_3	o_4	o_2	o_1

$\mathcal{D}$-version

Student side:

	1	2	3	4
o_1	d_4	d_3	d_2	d_1
o_2	d_1	d_4	d_3	d_2
o_3	d_1	d_2	d_3	d_4
o_4	d_4	d_1	d_2	d_3

Teacher side:

	1	2	3	4
d_1	o_1	o_4	o_3	o_2
d_2	o_3	o_4	o_2	o_1
d_3	o_3	o_2	o_1	o_4
d_4	o_3	o_4	o_2	o_1

When several matching exists, this yields alternative *valid partners* (e.g., here, d_2 and d_1 are valid partners for o_1).

Some basic properties

The algorithm just works... and at reasonable computational cost

Proposition (convergence)

The Gale-Shapley algorithm returns an allocation (i.e., a perfect matching).

Proposition (algorithmic complexity)

The algorithmic complexity of the Gale-Shapley algorithm is bounded, since it requires at most n^2 proposal loops. It generally ends much before that.

Proposition (stability)

The Gale-Shapley algorithm returns a stable allocation.

Optimality

It depends on the point of view...

Proposition (optimality – $\mathcal{O}$-version)

The $\mathcal{O}$-version of the Gale-Shapley algorithm returns

- *the* **best** *potential valid partners for the elements of $\mathcal{O}$,*
- *the* **worst** *potential valid partners for the elements of $\mathcal{D}$.*

We say that the allocation is $\mathcal{O}$-optimal and $\mathcal{D}$-pessimal.

Proposition (optimality – $\mathcal{D}$-version)

The $\mathcal{D}$-version of the Gale-Shapley algorithm returns

- *the* **best** *potential valid partners for the elements of $\mathcal{D}$,*
- *the* **worst** *potential valid partners for the elements of $\mathcal{O}$.*

We say that the allocation is $\mathcal{D}$-optimal and $\mathcal{O}$-pessimal.

This is a possibly surprising result...
However it also tells that, even if not optimal for some, it is stable! (and Pareto-efficient)

Incentive compatibility

Remember the definition of incentive-compatibility, from mechanism design

Definition (incentive compatibility in matching problems)

Given a marriage problem for 2 sets $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$ and $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$, the matching process is said to be *incentive-compatible* for $\mathcal{O}$ and/or $\mathcal{D}$ if it incentivises them to truthfully express their preferences.

If so, it is a (weakly-)dominant strategy to tell the truth.

Incentive compatibility

Remember the definition of incentive-compatibility, from mechanism design

Definition (incentive compatibility in matching problems)

Given a marriage problem for 2 sets $\mathcal{O} = \{o_1, o_2, \ldots, o_n\}$ and $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$, the matching process is said to be *incentive-compatible* for $\mathcal{O}$ and/or $\mathcal{D}$ if it incentivises them to truthfully express their preferences.

If so, it is a (weakly-)dominant strategy to tell the truth.

Proposition (incentive compatibility of Gale-Shapley algorithm)

The $\mathcal{O}$-version of the Gale-Shapley algorithm is incentive compatible for $\mathcal{O}$, but not for $\mathcal{D}$.
Conversely, the $\mathcal{D}$-version of the Gale-Shapley algorithm is incentive compatible for $\mathcal{D}$, but not for $\mathcal{O}$.

This is also a surprising result...! A consequence is that

- it is a dominant strategy for $\mathcal{O}$ (rep. $\mathcal{D}$) to express their true preferences
- it is **not** a dominant strategy for $\mathcal{D}$ (rep. $\mathcal{O}$) to express their true preferences

Wrapping up

- Stable matching is **mechanism design without money**
- In the general case, we look at **2 groups of players** that should be matched
- Matching should be **stable**, **optimal** and **incentive-compatible**
- The **Gale-Shapley algorithm** is the common approach to use

Further readings (/lecture notes for this topic):

Narahari Y (2014) *Game Theory and Mechanism Design* - chap. 31 (available on Blackboard Learn)

Thanks for your attention!